

[ACT] S-13

Protección de Rutas y Recursos por Rol

Tag Human Universal — Sistema de Acceso Residencial

Alumno	Oscar Espinoza Landeta
Docente	José Miguel Carrera Pacheco
Asignatura	Desarrollo Web y Programación (DWP)
Fecha	Abril 2026

1. ¿Qué se investigó?

Para completar esta actividad se investigaron cuatro conceptos fundamentales de seguridad en backend:

1.1 ¿Qué es un middleware?

Un middleware es una función que se ejecuta en medio del ciclo petición-respuesta de una aplicación Express. Recibe tres parámetros: req (petición), res (respuesta) y next (continuar la cadena).

En arquitectura de capas se visualiza así:

```
Cliente → [Middleware Auth] → [Middleware Rol] → [Controlador] → Respuesta

// Estructura básica de un middleware en Express
const miMiddleware = (req, res, next) => {
  if (!condicion) return res.status(401).json({ msg: "Bloqueado" });
  next(); // ✅ Pasar al siguiente eslabón
};
```

1.2 ¿Cómo se protegen rutas en backend?

En este proyecto se aplica una protección en tres capas (defense in depth):

- **Capa 1** — Verificación JWT: valida la firma criptográfica del token y que no haya expirado.
- **Capa 2** — Validación de sesión en BD: verifica que el JTI del token exista en la tabla sessions con is_active = TRUE. Permite revocar tokens antes de su expiración natural.
- **Capa 3** — Control de rol: verifica que el usuario autenticado tenga el rol requerido para la operación.

1.3 Errores de seguridad comunes

Error (OWASP)	Descripción	Mitigación en este proyecto
Broken Access Control	Rutas accesibles sin autenticación	sessionMiddleware en todas las rutas sensibles
Privilege Escalation	Usuario actúa con rol mayor al suyo	requireRole() verifica el claim role del JWT
JWT Algorithm None Attack	Firmar token con algoritmo "none"	jwt.verify() con secret forzado, rechaza alg:none
Token Replay	Reutilizar token después de hacer logout	JTI validado en BD; logout invalida el registro
SQL Injection	Injectar SQL en parámetros de la petición	Queries parametrizadas con \$1, \$2 en PostgreSQL

1.4 ¿Cómo se valida acceso por rol?

Se implementó un patrón factory middleware: una función que recibe los roles permitidos y devuelve el middleware correspondiente.

```
// roleMiddleware.js
const requireRole = (...roles) => (req, res, next) => {
  if (!req.user)
    return res.status(401).json({ code: "NOT_AUTHENTICATED" });
  // ... lógica de validación de roles ...
};
```

```
    if (!roles.includes(req.user.role))  
      return res.status(403).json({  
        code: "INSUFFICIENT_ROLE",  
        required: roles,  
        current: req.user.role  
      });  
  
    next(); //  Rol válido  
  };  
};
```

2. ¿Para qué? — Objetivos de Seguridad

La implementación de protección de rutas por rol garantiza tres propiedades fundamentales en el sistema:

- **Solo quien debe acceder, accede.** Un conductor (driver) no puede ver logs de auditoría. Un guardia (guard) no puede administrar usuarios del sistema.
- **Los datos están protegidos.** Los endpoints sensibles requieren sesión válida verificada en base de datos, no solo un token firmado.
- **El sistema es confiable.** Ante cualquier duda (token ausente, rol incorrecto, sesión revocada) el sistema rechaza el acceso. No existe "acceso por defecto".

Entregable	Propósito
roleMiddleware.js	Cerrar el acceso a operaciones según el rol del usuario
Rutas de admin protegidas	Demostrar separación real de privilegios entre roles
Corrección de auditoría	Eliminar la vulnerabilidad de ruta pública sin protección
Tests de seguridad (18 casos)	Verificar y documentar el comportamiento de cada escenario

3. ¿Qué se hizo? — Implementación

Se trabajó sobre la rama `feature/security-middleware` del repositorio. A continuación se detalla cada elemento implementado.

3.1 Nuevo middleware: `roleMiddleware.js`

Ubicación: `backend/src/middlewares/roleMiddleware.js`

Implementa el patrón `factory`: `requireRole(...roles)` devuelve un `middleware` que verifica que `req.user.role` esté dentro del conjunto de roles permitidos. Maneja dos casos de fallo independientes:

- **401 NOT_AUTHENTICATED** — Si `req.user` no existe (`sessionMiddleware` no corrió antes).
- **403 INSUFFICIENT_ROLE** — Si el rol del usuario no está en la lista de roles requeridos. La respuesta incluye el rol actual y los requeridos para facilitar el diagnóstico.

3.2 Corrección de vulnerabilidad: `auditoriaRoutes.js`

Ubicación: `backend/src/routes/auditoriaRoutes.js`

Se detectó que la ruta `POST /api/auditoria/registrar` estaba completamente pública: cualquier cliente sin autenticar podía registrar logs de acceso, fabricando entradas falsas en el historial del sistema.

```
// ❌ ANTES — Sin ninguna protección
router.post("/registrar", registrarLog);

// ✅ DESPUÉS — Autenticación + verificación de rol
router.post("/registrar",
  sessionAuth, // JWT válido + sesión activa en BD
  requireRole("guard", "admin"), // Solo guardias o admins
  registrarLog
);
```

3.3 Nuevas rutas de administrador: `adminRoutes.js`

Ubicación: `backend/src/routes/adminRoutes.js`

Se aplicó la protección a nivel de router completo con `router.use()`, de forma que todos los endpoints heredan la doble validación sin necesidad de repetirla en cada ruta.

```
router.use(sessionAuth); // Capa 1+2: JWT + sesión en BD
router.use(requireRole("admin")); // Capa 3: solo admin

GET /api/admin/users → Lista todos los usuarios del sistema
GET /api/admin/access-logs → Historial de accesos con JOINS
GET /api/admin/stats → Estadísticas por rol, estado y sesiones activas
```

3.4 Matriz de control de acceso

Endpoint	Sin token	driver	guard	admin
POST /api/auth/login	✅ Público	✅	✅	✅
GET /api/auth/me	❌ 401	✅	✅	✅

GET /api/sessions	 401			
POST /api/auditoria /registrar	 401	 403		
GET /api/admin/use rs	 401	 403	 403	
GET /api/admin/acc ess-logs	 401	 403	 403	
GET /api/admin/sta ts	 401	 403	 403	

4. Tests de Seguridad — Evidencia de Bloqueo

Se creó el archivo `backend/tests/security.test.js` con 18 casos de prueba organizados en 6 suites. Los tests son unitarios con mocks y no requieren una base de datos activa.

Suite	Descripción	Casos	Resultado esperado
SEGURIDAD 1	Acceso sin token	2	401 NO TOKEN
SEGURIDAD 2	Token inválido / expirado	3	401 TOKEN_INVALID / TOKEN_EXPIRED
SEGURIDAD 3	Rol insuficiente	4	403 INSUFFICIENT_ROLE
SEGURIDAD 4	Acceso con rol correcto	4	<code>next()</code> invocado, sin error
SEGURIDAD 5	Cadena <code>sessionAuth</code> → <code>requireRole</code>	2	Comportamiento integrado
SEGURIDAD 6	Claims del JWT	3	role, jti presentes; sin datos sensibles

4.1 Resultado de ejecución

```
npm test (desde el directorio backend/)

PASS tests/security.test.js (12.185 s)
PASS tests/session.test.js

  SEGURIDAD 1 – Acceso sin token debe ser bloqueado (401)
    ✓ rutas protegidas devuelven 401 cuando no hay Authorization header
    ✓ ruta con token vacío devuelve 401
  SEGURIDAD 2 – Token inválido debe ser bloqueado (401)
    ✓ token con firma incorrecta devuelve 401 TOKEN_INVALID
    ✓ token malformado (string basura) devuelve 401
    ✓ token expirado devuelve 401 TOKEN_EXPIRED
  SEGURIDAD 3 – Rol insuficiente debe ser bloqueado (403)
    ✓ driver no puede acceder a rutas solo para admin
    ✓ driver no puede registrar logs de auditoría (solo guard/admin)
    ✓ guard no puede acceder a rutas exclusivas de admin
    ✓ usuario sin req.user en requireRole devuelve 401
  SEGURIDAD 4 – Acceso permitido para el rol correcto
    ✓ admin puede acceder a rutas de admin
    ✓ guard puede acceder a rutas de guard y admin (multi-rol)
    ✓ admin también puede acceder a rutas de guard
    ✓ driver puede acceder a rutas exclusivas de driver
  SEGURIDAD 5 – Cadena: sessionAuth → requireRole
    ✓ token de admin válido + requireRole admin → acceso concedido
    ✓ token de driver válido + requireRole admin → 403 bloqueado
  SEGURIDAD 6 – Claims del JWT
    ✓ JWT generado al login contiene campo role
    ✓ JWT contiene jti único para cada emisión
    ✓ JWT no contiene password_hash ni datos sensibles

Test Suites: 2 passed, 2 total
Tests:      32 passed, 32 total ← 18 de seguridad + 14 de sesiones
```

4.2 Evidencia de bloqueo de accesos indebidos

Respuestas HTTP reales del sistema ante intentos no autorizados:

Sin token — POST /api/auditoria/registrar

```
HTTP 401
{ "msg": "Acceso denegado. No hay token." }
```

Token de driver — POST /api/auditoria/registrar (requiere guard/admin)

```
HTTP 403
{ "msg": "Acceso denegado. Se requiere rol: guard o admin. Tu rol actual es:
driver.",
  "code": "INSUFFICIENT_ROLE",
  "required": ["guard","admin"],  "current": "driver" }
```

Token de guard — GET /api/admin/users (requiere admin)

```
HTTP 403
{ "msg": "Acceso denegado. Se requiere rol: admin. Tu rol actual es: guard.",
  "code": "INSUFFICIENT_ROLE",
  "required": ["admin"],  "current": "guard" }
```

Token expirado — cualquier ruta protegida

```
HTTP 401
{ "msg": "Tu sesión ha expirado. Por favor inicia sesión nuevamente.",
  "code": "TOKEN_EXPIRED" }
```

Token de admin — GET /api/admin/users (acceso concedido)

```
HTTP 200
{ "users": [...], "total": 5 }
```


5. Aprendizaje

Esta actividad permitió consolidar competencias de ingeniería backend que van más allá de hacer funcionar una aplicación — se trata de hacerla segura y confiable.

► Cerrar el sistema correctamente

El código funcional no es suficiente. Una ruta que "funciona" pero es accesible sin autenticación es una vulnerabilidad real, como se demostró con `/api/auditoria/regarar`. Cerrar el sistema implica revisar cada endpoint y preguntar: ¿quién debería poder llamar esto?

► Implementar control de acceso real

La diferencia entre autenticación (¿eres quien dices ser?) y autorización (¿tienes permiso para esto?) se vuelve concreta al implementar el middleware `requireRole`. No basta con tener un JWT válido; el rol del usuario determina qué operaciones puede ejecutar.

► Pensar como ingeniero backend

Se aplicaron principios de seguridad estándar de la industria: principio de mínimo privilegio (cada rol solo puede lo necesario), defensa en capas (JWT + sesión en BD + rol), y fail-secure (ante duda, rechazar). Estos principios se usan en sistemas reales de producción.

► Valor de los tests de seguridad

Los tests de seguridad no prueban que el código funciona — prueban que el código falla correctamente ante ataques. Los 18 casos escritos documentan el comportamiento del sistema ante cada tipo de acceso indebido y sirven como regresión: si alguien rompe la seguridad en el futuro, los tests lo detectarán.

► Códigos de error como contrato de API

Usar códigos semánticos (`INSUFFICIENT_ROLE`, `TOKEN_EXPIRED`, `SESSION_INVALID`) en lugar de mensajes genéricos permite que el frontend tome decisiones inteligentes: saber si debe redirigir al login o mostrar un "acceso denegado", sin exponer detalles internos del sistema.